

AI 工具使用说明

一、使用的工具

环节	工具	用途
主力开发	Claude Code（命令行 Agent）	生成模拟数据脚本、看板页面、文档初稿；在无头 Chromium 里自查渲染结果
渲染自查	Playwright + Chromium（由 Claude Code 驱动）	打开页面截图、抓 console 报错、抽取页面上的 KPI 文本做断言
交叉校验	Python（自己写的独立脚本）	绕开前端逻辑，独立算一遍所有指标，和页面显示值逐项比对
图表库	ECharts 6.1.0（本地 vendor，无 CDN 依赖）	全部图表

没有使用任何低代码/拖拽式 BI 工具，看板是手写的单页应用，所有聚合逻辑可读可查。

二、关键提示词

按实际使用顺序，保留了主干：

- 需求拆解：把题目原文交给 Claude Code，要求先不写码，列出交付物清单、技术选型的取舍（单文件 HTML vs 前端工程 vs Streamlit），并明确指出评审方要"双击能打开"，让它据此推荐方案。
- 数据建模：
 - > 生成 60 天 × 8 个点位的模拟数据，固定随机种子保证可复现。刻意埋几条能被发现的运营故事：某设备整机离线两天、某点位周末排队导致生成时长翻倍、某点位审核拒绝率长期偏高、某设备最近几天模型报错率上升、一次线下活动带来参与人数峰值、以及一台设备此刻仍然离线。
 - 指标口径：
 - > 平均生成时长要能跨天跨点位正确聚合；P95 不能用"每天的 P95 再平均"。给出一个在任意筛选切片下都算得准的存储方案。
 - 看板实现：
 - > 一行筛选控件作用于整页；6 个 KPI 磁贴带环比和迷你趋势线；图表覆盖趋势、点位对比、转化漏斗、审核构成、高峰时段；设备状态面板；异常事件表可按级别/类型/状态筛选并展开处理建议。不允许双轴图；标称类别不做明暗渐变；每张图配一个表格视图。
 - 自查：
 - > 用无头浏览器打开页面，报告 console 错误，抓取 KPI 文本，再用 Python 独立算一遍同样口径的指标，逐项比对，不一致就指出来。

三、本人完成的主要修改

AI 的初稿能跑，但跑得通不等于对。下面是我自己发现并改掉的问题，按性质分三类。

1. 指标口径错误（影响数字正确性）

- 平均生成时长把失败请求也算了进去。失败多半是超时，把它们计入均值会把数字系统性拉高。我改成只统计成功生成，并在指标说明里写明这条口径。
- P95 原本按天存一个值，聚合时再求平均——分位数不能这么算。我改成每条记录存一个 14 档的生成时长直方图，跨切片时先合并直方图再线性插值。这样筛任意日期 + 任意点位，P95 都是准的。
- 转化率的分子分母混了口径。生成成功率是"次数"口径（一人可以生成多次），扫码/分享转化率是"人数"口径。我在数据里拆成两组字段，并在文档里逐条标注。

2. 运营视角的缺陷（数字对，但没用）

- 告警按天刷屏。初版对每个越界的日子都生成一条事件，一个持续 60 天的问题产生了 60 条记录，异常列表完全没法看。我改成按连续区间去重：一次持续异常只开一张单，详情里写持续天数——这也是真实告警系统的做法。
- 30 天平均把正在发生的事故抹平了。有个点位最近 5 天成功率跌到 84%，但 30 天聚合后是 94%，看板一条预警都不报。我给规则引擎加了第二层判定：区间合计和范围内最后一天各判一次，只要最后一天越界就标记"仍在发生"并排到最前。这个改动让实时离线的设备和持续衰退的设备都浮了上来。
- 预警的处理建议给错了。"扫码转化率偏低"复用了"素材下载失败"的处置文案，让人去查 CDN——但转化率低是现场引导问题，不是设备故障。我给这条规则单独写了建议动作。

3. 图表渲染缺陷 (AI 自己看不见的)

这几个问题只有真的把页面打开看一眼才会发现，AI 写完代码是不会自己去看的：

- 百分比坐标轴显示成了 0.97、0.96。原因是配置合并顺序写反了，公共轴配置覆盖掉了自定义的格式化函数。
- 阈值虚线根本不在可视范围内。坐标轴自动缩放到数据范围（91%–97%），而阈值是 90%，线被画到了画布外面——卡片副标题还写着“虚线以下为低于阈值的日子”。我改成让轴的上下端点强制包含阈值。
- 坐标轴端点印出了 **12.471364357323855s**。加留白后没取整。改成向外取整到干净的刻度。
- 两条趋势线几乎完全重合，末端标签还互相压字。参与人数和生成请求次数本来就高度相关，同时画出来信息量接近零。我换成“参与人数 vs 扫码带走人数”——两条线量级拉得开，而且它们之间的距离直接就是流失掉的人，这才是运营想看的。
- 末端标签被卡片右边界切掉。去掉了逐条末端标签，改成只标一个峰值，身份识别交给图例。

4. 其他工程决策

- `file://` 下 `fetch()` 会被 CORS 拦，评审方双击打开会白屏。数据生成脚本同时产出一份 `mock.js`（挂到 `window.MOCK_DATA`）用 `<script>` 引入，`mock.json` 作为交付物本身保留。ECharts 也放本地 `vendor/`，全程不依赖外网。
- 配色不靠眼睛判断。分类色和漏斗的序数色都跑过对比度与色盲可分辨性校验；状态色（通过/拒绝/在线/离线）一律配文字标签，不让颜色单独承担含义；每张图都有表格视图，数值不依赖悬浮提示。
- 阈值集中在数据的 `meta.thresholds` 里，看板和指标说明文档引用同一份，避免两边写着不同的数。

四、AI 帮了什么，没帮上什么

帮上的：把结构性的活儿压缩得很厉害——数据生成脚本、页面骨架、图表配置、文档初稿，这些如果手写要花掉大半天。让它顺手写一个 Playwright 自查脚本也很划算。

没帮上的：所有需要业务判断的地方它都会给一个“看起来合理”的答案，但未必对——平均时长算不算失败请求、告警要不要抑制、30 天平均会不会掩盖问题，这些只有真的想过运营每天怎么用这块看板才知道。渲染层面的问题它更是完全看不见，必须自己打开页面逐张图看。

我的用法是：让 AI 写第一版，我负责判断这一版哪里是错的。上面第三节那几个渲染缺陷，就是这么找出来的。